DEVGUARD

DevGuard

A safety net that stops accidental Git pushes of company code to the wrong place, and shows you what happened.

Developer & administrator guide | Laptop agent | Control server (API) | Dashboard

PREPARED BY

Tushar Saxena

Hackathon build · October 2026

Plain-language documentation

CONTENTS

What is inside

1 **What DevGuard is** 3
The problem it solves and what it promises (and does not)

2 **How the three parts fit together** 4
Agent, API and dashboard, and the life of one push

3 **Part 1: the laptop agent** 5
The command-line tool and Git hook on each developer's laptop

4 **Part 2: the control server (API)** 6
The single source of truth for rules, events and approvals

5 **Part 3: the dashboard** 7
The web screen where administrators work

6 **What DevGuard supports** 8
Checks, modes, destinations, exceptions, alerts, platforms

7 **Getting started** 9
Step by step for the administrator and for the developer

8 **Everyday use** 11
A blocked push, exceptions, reading events, lost laptops

9 **Running DevGuard** 12
Quick start, ports, settings and email alerts

10 **Limits and security notes** 13
What it cannot do, stated honestly

11 **Quick reference and troubleshooting** 14
Commands, fixes and a glossary

SECTION 1

What DevGuard is

DevGuard protects company code from **honest mistakes**. Before a push leaves a company laptop, it checks where the code is going and what is inside it, tells the developer, alerts the right people, and keeps a record of everything.

The problem

> **Developers push code from company laptops every day. One mistaken push can put a whole project on a personal account, and one committed key can hand over access. Yet the company cannot stop it before it happens, cannot see it when it does, and has no evidence afterwards.**

WRONG DESTINATION

A project is pushed to a personal, look-alike or mistyped remote, and the whole codebase is outside the company.

SECRETS IN COMMITS

A key, token or `.env` file is committed. Deleting it in a later commit does not remove it from history, so it travels with every push.

NO VISIBILITY

Nobody sees who pushed what, where and when. Problems surface days later, by accident, if at all.

NO PROOF AND NO PROCESS

Exceptions are agreed informally. There is no record of who approved what, for how long, or why.

How DevGuard answers it: prevent, detect, notify, prove

- **Prevent.** Blocks a push to an unapproved destination, or one carrying prohibited files or secrets, *before* anything leaves the laptop.
- **Detect.** Scans every outgoing commit, including deleted history, for secrets, keys and `.env` files.
- **Notify.** Tells the developer why in plain words, with a one-line way to ask for an exception, and emails security administrators about blocked pushes and secret findings.
- **Prove.** Records every push decision as an event and every administrator change in an append-only history. Exceptions carry a reason, an approver and an expiry.

DEVGUARD IS

A safety net against **mistaken** pushes, with fixed, predictable rules (no guessing, no AI deciding), clear messages and a full record.

DEVGUARD IS NOT

A monitor of everything employees do, complete data-loss prevention, or a tamper-proof lock. It covers Git pushes, and a determined person can bypass a local hook.

WHO DOES WHAT

Administrators decide which projects are protected, where they may go and who is covered, all from the dashboard. **Developers** install the tool and enroll **once**; after that DevGuard only speaks up when something is blocked.

SECTION 2

How the three parts fit together

DevGuard is made of three separate programs. Each has one job, and they talk in a fixed pattern. The agent keeps working without the dashboard, and for a limited time even without the server: the server is the authority, the agent is the enforcer.

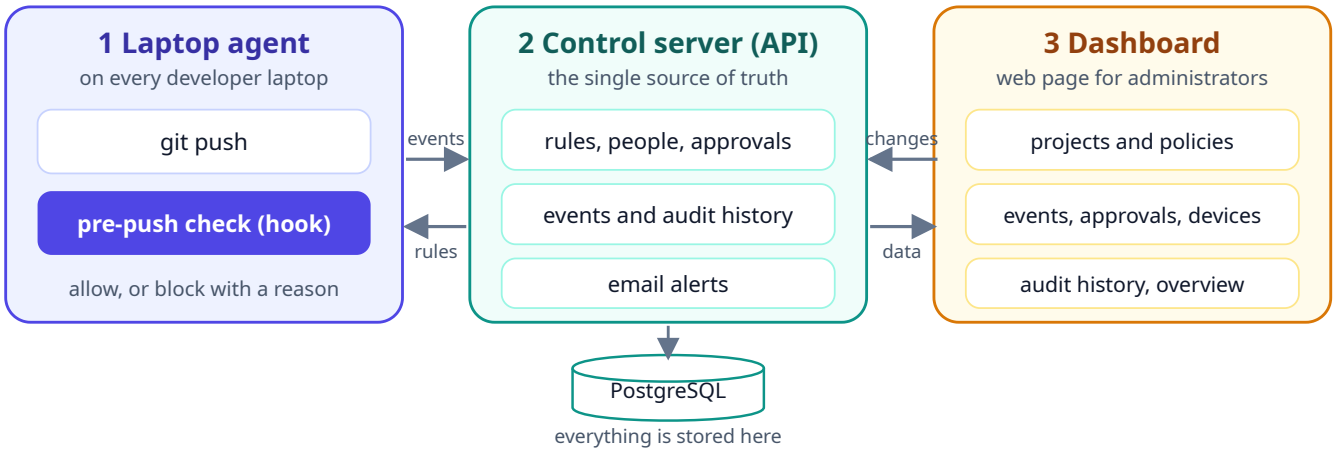


Figure 1. The three parts. The agent and the dashboard never talk to each other; they both talk to the API.

1

Laptop agent

Enforces. Sits in Git on the developer's laptop and decides allow or block at push time.

2

Control server

Decides and remembers. Holds the rules, the people, the approvals and the history.

3

Dashboard

Shows and controls. Where administrators set rules and review what happened.

The life of one push

1. **The developer runs `git push`.**
   Git runs the DevGuard hook just before sending anything.
2. **The agent works out whether the repository is protected.**
   It matches the repository to a project the administrator assigned to this person. No match means the repository is not protected and the push goes through.
3. **It checks the destination.**
   The remote address is cleaned up and compared exactly with the project's approved destinations.
4. **It checks the commits being sent.**
   Each outgoing commit is scanned for prohibited files and known secret patterns, including ones added and later deleted.
5. **It decides.**
   In *enforce* mode a problem blocks the push and explains why. In *audit-only* mode the push is allowed and the problem is only recorded.
6. **It reports.**
   An event goes to the control server (queued on the laptop if offline). Administrators see it on the dashboard, and serious ones can trigger an email.

PART 1 OF 3

The laptop agent

A small command-line program called `devguard` (Node.js and TypeScript, no window or tray icon), plus a Git hook it installs. This is the part developers meet.

ITS JOB

- Be the **enforcer**: allow or block each push.
- Recognise which repositories belong to which protected project.
- Keep a saved copy of the rules, and queue events to send when it can.
- Let the developer ask for an exception and check its status.

WHAT IT NEVER DOES

- It never decides policy. The server is the source of truth.
- It never sends file contents or secret values: only the rule, kind of secret, file path, line and commit.
- It never changes your code or Git history, and watches only Git pushes.

What a developer sees

A normal push is **silent**. A blocked push prints a plain message in the terminal (or the IDE's Git panel):

```
$ git push fork main
DevGuard blocked this push.
  Project: Portal (confidential, enforce)
  Destination: github.com/personal/fork
  Why: The push destination is not approved for this project.
  To ask for an exception to this destination:
    devguard request-exception --destination https://github.com/personal/fork --reason "why you need it"
  Reference: 3f9c2e1a-....
```

The remote is untouched. The **Reference** lets an administrator find exactly this push on the dashboard. If a secret or prohibited file caused the block, the message names the file, line and rule, never the secret itself.

Commands

Command	What it does
`devguard enroll --server <url> --token-stdin`	One-time setup: registers the laptop, installs the Git hook, downloads your projects.
`devguard status [--check-server]`	Enrollment, hook state, saved rules and their age, queued events, and whether this repository is protected.
`devguard sync`	Fetch the latest rules and send any queued events now.
`devguard request-exception` `devguard exception status`	Ask an administrator to allow a blocked destination, with a reason. Then see your requests: waiting, approved (until when), rejected, expired or revoked.
`devguard register --project <id>`	Link a repository to a project by hand when automatic matching cannot.
`devguard install / uninstall`	Add or remove the hook. Uninstall restores your previous Git setup.

PART 2 OF 3

The control server (API)

The brain and the memory of DevGuard. Every rule, person, approval and event lives here, and every request to change something is checked here.

ITS JOB

- **Sign-in and permissions** for administrators, and a separate secret for each laptop.
- **Projects and rules:** approved destinations, canonical repositories, versioned policies, who is covered.
- **Devices:** issue one-time enrollment tokens, enroll laptops, revoke lost ones.
- **Events:** receive what the agents report and keep it for review.
- **Exceptions:** take requests, let a *different* administrator approve, reject or revoke, and check scope and expiry every time.
- **Audit history and numbers** for the dashboard.
- **Email alerts** to administrators for serious events.

RULES IT ENFORCES

- Every protected action is permission-checked on the server. Hiding a button is never the security.
- Approving an exception needs a second administrator. Nobody approves their own request.
- Approvals are checked for project, destination and expiry every time, never trusted from the laptop.
- Secrets for laptops and tokens are stored only as hashes.
- Events cannot carry file contents: the server rejects any field that could.
- The audit history can only be added to. It cannot be edited or deleted.

What it keeps

Kind of information	Examples
People and laptops	Users, their roles, enrolled devices, last time each laptop was seen.
Protection rules	Projects, how sensitive each is, approved destinations, policy versions (enforce or audit-only, prohibited files, secret rules), who is covered.
What happened	Push events (decision, rule, destination, time) and exception requests with their decisions.
History	Every administrator change: who, what, when, before and after.

FOR DEVELOPERS AND OPERATORS

Built with **NestJS** and **Prisma** on **PostgreSQL**. It follows a controller, service, repository layering, validates every request against shared schemas, and publishes an interactive API reference at `/docs` when not running in production. Permissions are two kinds today: **security administrator** and **employee** (plus the per-laptop credential). Per-project team-lead roles are not built yet.

ALERTS ARE A MINIMAL FIRST VERSION

The server emails active security administrators when a push is blocked in enforce mode, or an event reports a secret or a scan error. It is best-effort: no retries, no inbox, no Slack or Teams, and a compromised laptop could send many mails. See section 9.

PART 3 OF 3

The dashboard

A web page where security administrators set the rules and review what happened. It makes no security decisions itself: the server decides what each person may do.

ITS JOB

- Give administrators one place to define projects, destinations, policies and people.
- Issue enrollment tokens and revoke devices.
- Review events and approve or reject exception requests.
- Show the audit history and a summary of activity.

The screens

Screen	What you do there
Overview	Pick a period and see blocked, allowed, allowed-by-exception and could-not-check pushes; top rules; blocks per day; pending approvals; exceptions in force; active, stale and revoked laptops.
Events	Browse every push attempt the hooks reported. Filter by decision or rule. Paste a **Reference** from a developer to find that exact push. Details show the rule, mode, policy version and, for secrets, the kind, file and line, never the value.
Approvals	Review pending exception requests: give a reason and an expiry, then approve or reject. Revoke an active approval immediately.
Projects	Create projects. For each: approved destinations, canonical repositories, policy (mode, file patterns, secret rules), people covered, and the repositories laptops have matched to it.
Devices	See each enrolled laptop and when it was last seen, issue one-time enrollment tokens (with copy buttons), add a person, revoke a device.
Audit	An append-only list of every administrator change, filterable by kind of change or target.

Works on phones

Below 1024 pixels wide it switches to a menu drawer and card lists, with large tap targets. No sideways scrolling from 360 pixels up.

Light and dark

System, light or dark theme, remembered in your browser, with readable contrast in both.

Accessible

Skip link, keyboard-friendly menus and dialogs, a page title per screen, and clear messages next to form fields.

Built with React, Vite and Tailwind CSS. Today only security administrators can sign in; other people are told the dashboard is for administrators. Open it at `http://localhost:5173` in development.

SECTION 6

What DevGuard supports

A plain list of what works today, so there are no surprises.

Checks on every protected push

Check	How it works
Approved destination	The remote must match an approved destination for the project, by **exact host and repository path**. Look-alikes such as `github.com.evil.io` or `github.com@evil.io` do not match. A remote that cannot be understood is treated as **not approved**.
Remote address forms	HTTPS (`https://github.com/acme/portal`), SSH (`ssh://git@host/owner/repo`) and scp-style (`git@github.com:acme/portal.git`). Different spellings of the same repository are treated as the same.
Prohibited files	Default patterns: `.env` and `.env.*` (but not `.env.example`), `*.pem` , `*.key` , `*.p12` , `*.pfx` , SSH private keys (`id_rsa*` , `id_ed25519*` , `id_ecdsa*` , `id_dsa*`), `.npmrc` , `credentials.json` , `*.kdbx` . Administrators can edit the list per project.
Secret patterns	High-confidence patterns only: private key blocks, AWS access key IDs, GitHub tokens, Slack tokens, and common provider tokens (Stripe live keys, Google API keys, npm tokens, GitLab tokens, SendGrid keys).
History, not just files	It scans the **commits being pushed**, so a secret added and then deleted in a later commit is still caught. It also handles new branches, several branches at once, force-pushes, branch deletions and tags.

Modes, exceptions and offline behaviour

Feature	How it works
Enforce mode	A problem **blocks** the push and explains why.
Audit-only mode	The push is **allowed**, and DevGuard records that it would have been blocked. Good for trying a policy before enforcing it.
Exceptions	Scoped to **one project and one destination**, with a reason and an expiry (at most 72 hours). A different administrator must approve. An exception never waives a secret or prohibited-file block.
Server unreachable	The laptop uses its last saved rules. If they are older than the limit (default 60 minutes): enforce-mode projects **block**; audit-only projects allow and record. Events wait in a local queue and are delivered later. Exceptions cannot be checked offline.
Never fails open	If DevGuard cannot read its own state, cannot parse the destination, or hits an internal error in enforce mode, it blocks and says why. Only a repository that matches no project is allowed through.

Features and platforms

Capability	Status	Notes
Block unapproved destinations	**Supported**	Per project, from the dashboard.
Scan outgoing commits	**Supported**	Prohibited files and the secret patterns above.
Audit-only trial mode	**Supported**	Per project policy.
Exception request and approval	**Supported**	Two administrators needed.
Event review and metrics	**Supported**	Overview, Events and Audit screens.
Revoke a lost laptop	**Supported**	Stops sync, events and exceptions for that laptop.
Email alerts	**Minimal**	Administrators only, best-effort, no retries.
Team-lead and project-owner roles	**Not yet**	Only security administrators and employees exist.
Notification inbox, Slack, Teams	**Not yet**	Planned, not built.
Installer, auto-update, MDM rollout	**Not yet**	Install from the repository today.
Linux laptops	**Supported**	Built and tested on Linux. macOS is untested; Windows is out of scope.

SECTION 7

Getting started

Two short jobs: the administrator prepares a project and a token, then the developer enrolls once.

Administrator

Prepare a project (once per project)

1. **Sign in to the dashboard and open Projects.**
   Create the project and choose how sensitive it is.
2. **Add the approved destination.**
   For example `https://github.com/acme/portal`. Any spelling works; `git@github.com:Acme/Portal.git` is stored as `github.com/acme/portal`. Add the same address under **Canonical repositories** so laptops can recognise the project by its remote.
3. **Choose the policy and save it.**
   Pick **Enforce** (block) or **Audit only** (allow and record), review the file patterns and secret rules, then **Save new version**.
4. **Choose who is covered.**
   Tick the people whose laptops should get these rules and **Save people**. Only people listed here receive the project.

Administrator

Issue an enrollment token (once per laptop)

Open **Devices**, choose the person under *Enroll a laptop*, and press **Issue token**. Use **Copy command** to get a ready-to-paste line. The token is shown **once**, works **once**, and expires in **30 minutes**. Send it to the person through a private channel.

Developer Install and enroll (once)

You need Node.js 22 or newer, pnpm 10 or newer and Git 2.30 or newer. There is no installer yet, so the tool is installed from the repository:

```
# 1. get and build the tool
git clone <the DevGuard repository> && cd dev-guard
pnpm install
pnpm --filter @devguard/shared build
pnpm --filter @devguard/policy-engine build
pnpm --filter @devguard/agent build
mkdir -p ~/.local/bin && ln -sf "$PWD/apps/agent/dist/main.js" ~/.local/bin/devguard

# 2. enroll with the token you were given (once)
echo '<token>' | devguard enroll --server https://devguard.example.com --token-stdin

# 3. check it worked
devguard status --check-server
```

A healthy result says `Device: enrolled`, lists your project under cached policies, `Global hook: installed`, and `Server check: reachable`. Using `--token-stdin` keeps the token out of your shell history. The server address must be `https` (plain `http` is only accepted for `localhost`).

> **KEEP THE FOLDER WHERE IT IS**
>
> The hook remembers where Node and the agent live. If you move the cloned folder or remove that Node version, every push is **blocked** (on purpose) until you run `devguard install --global` again.

After enrolling there is nothing more to do. Rule changes arrive automatically at your next push, or when you run `devguard sync`. To remove DevGuard: `devguard uninstall --global`.

SECTION 8

Everyday use

When a push is blocked: asking for an exception

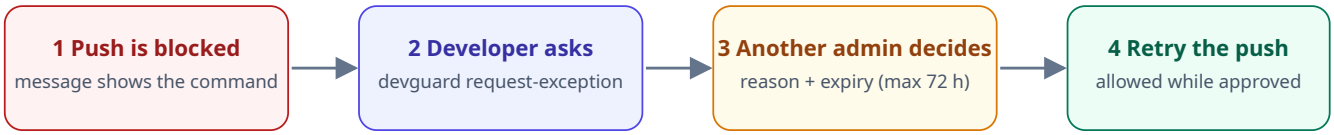


Figure 2. The exception flow. The server checks project, destination and expiry on every push.

```
$ devguard request-exception --destination https://github.com/personal/fork --reason "Handing the
prototype to the customer"
Requested an exception for Portal to push to github.com/personal/fork.
  Request id: ...
  An administrator has to approve it in the dashboard. Then run your push again.

$ devguard exception status
  Status: approved until 2026-10-20 18:00   (or: waiting, rejected, expired, revoked)
```

- An exception covers **one project and one destination** for a limited time. It does not unlock anything else.
- It never overrides a block caused by a **secret or a prohibited file**. Remove that content from the commits first.
- It only works while the laptop can reach the server, because the server checks it each time.

Reading events on the dashboard

> **READ EVENTS HONESTLY**
>
> An event says the hook **saw a push attempt and made a decision**. It does not prove that data left, or did not leave, the laptop. "No event" can also mean the push skipped the hook, the laptop is not enrolled, or events are still queued offline. Check **Last seen** on the Devices screen before concluding anything.

- Use the **Reference** a developer quotes to jump straight to their push.
- A laptop whose *Last seen* is old is not reporting. Ask the owner to run `devguard sync` and `devguard status`.
- Use **Audit** to see who changed a policy or approved an exception, and when.

Changing who is covered, and lost laptops

- **Add or remove a person** on the project's *People covered* list. It takes effect at their next push or sync.
- **Lost or retired laptop:** press **Revoke** on the Devices screen. The dashboard asks you to confirm and names the device. A revoked laptop can no longer sync, send events or use exceptions.
- **A repository that is not recognised** (for example a fork with no matching remote): `devguard register --project <id>` inside that repository.

SECTION 9

Running DevGuard

For the person who sets up the server and dashboard. The laptop agent is never run in a container; it runs on each laptop.

Quick start with Docker

```
# one-time: copy the example settings and fill in secrets (never commit .env)
cp .env.example .env

# start the database, mail catcher, API and dashboard
docker compose up -d --build

# load demo users and data (runs on the host against the database)
set -a; . ./.env; set +a
pnpm install && pnpm --filter @devguard/shared build
SEED_PASSWORD='choose-a-12+-character-password' pnpm --filter @devguard/api seed
```

Then open the dashboard at `http://localhost:5173` and sign in as `admin@example.test` with the password you chose. The demo data is synthetic and meant for trying things out.

> **DEVELOPMENT STACK, NOT PRODUCTION**
> The containers are for local use and bind to your own machine only. A real deployment needs HTTPS in front of the dashboard and API, production settings, and a hardened image.

Addresses and ports

What	Address	Notes
Dashboard	`localhost:5173`	Also proxies the API to the browser, so sign-in just works.
API	`localhost:3000`	Change with `API_PORT` if something else uses 3000. Reference at `/docs`, health at `/health`.
Database (PostgreSQL)	`localhost:5433`	5433 on purpose, because 5432 is often taken.
Mail catcher (Mailpit)	`localhost:8025`	Web inbox for alert emails in development. SMTP on 1025.

Important settings

Setting	Meaning
`JWT_SECRET`	Required. At least 32 random characters. The API refuses to start without it.
`DATABASE_URL`	Required. The PostgreSQL address.
`DASHBOARD_BASE_URL`	Where the dashboard is served. Used for links in emails and for the browser access rules.

Setting	Meaning
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`	Email alerts. Leave `SMTP_HOST` empty to turn alerts off. Passwords are only used over an encrypted connection.
`TRUST_PROXY_HOPS`	How many reverse proxies sit in front of the API (0 by default, 1 in the Docker setup) so rate limits see the real caller.
`DEVGUARD_POLICY_MAX_STALE_MINUTES`	On a laptop: how long saved rules stay trusted without reaching the server (default 60, never more than 24 hours).

Email alerts (minimal)

When `SMTP_HOST` is set, active security administrators get an email when a push is **blocked in enforce mode**, or when an event reports a **secret or a scan error**. Audit-only events never send mail. The email contains the project, person, laptop, destination, rule names, outcome and time, plus a link to the dashboard. **It never contains file paths, contents or secrets.** Delivery is best-effort: failures are logged and dropped, never retried, and never change what happens to a push.

SECTION 10

Limits and security notes

DevGuard is honest about what it cannot do. Please read this section before relying on it.

What DevGuard cannot stop

Way around it	Why it is possible
`git push --no-verify`, deleting the hook	A hook on a person's own laptop can be skipped or removed. DevGuard is not tamper-proof.
A repository-level `core.hooksPath` (for example husky)	It can override the global hook. DevGuard detects and reports this; it cannot prevent it.
Other Git clients, other machines	Only the hook on enrolled laptops is covered.
Anything that is not a Git push	Browser uploads, USB, email, cloud sync, copy and paste, screenshots.
A repository never matched to a project	Unmatched repositories are **not protected**. This is documented behaviour, not an oversight.
An SSH alias in `~/.ssh/config`	An alias that points an approved-looking name at another server cannot be seen from the URL.
Secrets the detector does not know	The secret detector is small and deliberate. It is not comprehensive: no passwords, no random-looking strings, no obfuscated secrets.

Principles it is built on

Fail safe
When it cannot tell, it treats the push as unapproved and explains why. It never quietly lets something through.

Exact matching
Destinations are compared by exact host and repository path. Look-alike addresses never match.

No contents, ever
Only the rule, kind of secret, file path, line and commit leave the laptop. Never the file or the secret.

The server decides
The API checks permissions, scope and expiry on every request. Nothing the laptop or the browser claims is trusted.

Two people for exceptions
An approver is never the requester, and the approver must still be an active administrator when the exception is used.

History you can trust
Administrator changes are written to an append-only record that cannot be edited or deleted.

Known gaps in this build

- A compromised laptop can trigger many alert emails. A per-laptop limit or a digest email is the planned fix.
- Signing out clears the browser cookie, but a copied session stays valid until it expires (60 minutes by default).
- An exception requested for the `ssh://host:443` form of an address will not match the https form of the same repository.
- Linux only for now. Installation is manual and there is no auto-update.

SECTION 11

Quick reference and troubleshooting

Troubleshooting

Symptom	What to try
A push that should be blocked is allowed	The repository may match no assigned project (check *People covered* and the canonical repository), the hook may be missing (`devguard status`), or `--no-verify` was used.
Every push is blocked with "agent was not found"	The folder or Node version moved. Run `devguard install --global` again.
Blocked with "policy out of date"	The laptop has been offline too long. Reconnect (VPN or network) and run `devguard sync` .
"Enrollment token is invalid, expired or already used"	Tokens work once and last 30 minutes. Issue a new one on the Devices screen.
"must be https" when enrolling	Use an `https` server address. Plain `http` only works for `localhost` .

Symptom	What to try
The dashboard says it is for administrators	You signed in as a non-administrator. Use an administrator account.
API will not start on port 3000	Another program may use it. Set `API_PORT` in `.env`.
No alert emails arrive	Check `SMTP_HOST` is set and look in the Mailpit inbox at `localhost:8025`. Audit-only events do not send mail.

Glossary

Agent
The `devguard` program and Git hook on a laptop.

Audit-only
A mode that allows pushes but records what would have been blocked.

Canonical repository
The address that lets a laptop recognise which project a repository belongs to.

Enforce
A mode that blocks pushes that break the rules.

Enrollment token
A one-time code, valid for 30 minutes, that links a laptop to DevGuard.

Event
A record that the hook saw a push attempt and what it decided.

Exception
A time-limited approval for one project and one destination.

Hook
A small script Git runs automatically at a set moment. DevGuard uses the *pre-push* hook.

Policy
A project's rules: mode, prohibited files and secret rules. Each save creates a new version.

Reference
The short id shown in a block message, used to find that push on the dashboard.

WHERE TO READ MORE

The repository's `docs` folder has the full guides: `usage.md` (running it for a team), `testing/how-to-test.md` (a hands-on walkthrough with a throwaway laptop), `laptop-tool.md`, the architecture notes and the decision records (ADR files).

DevGuard

A safety net for honest mistakes, with a clear explanation when it steps in and a record of everything it did.

It prevents mistakes in ordinary Git workflows through fixed, predictable rules. It does not claim to prevent every way data could leave a laptop.

Prepared by **Tushar Saxena**

Hackathon build · October 2026 · Documentation generated from the project's own docs and source.